

5교시: 장애와 Metric 연결

W4D3 Lesson 5

failure and metric correlation

kubernetes

Prometheus

Grafana

kubectl

W

N

E

S

실제 원인과 메트릭의 상관관계를 연결하여 빠르게 원인을 파악하고 개발자에게 정확히 인계하자!

1. 실제 시나리오별 메트릭 → 증거 → 원인 상관관계

1 CrashLoopBackOff

재시작 횟수 증가

```
increase(kube_pod_container_status_restarts_total{namespace="app", pod="api"}[5m])
```

컨테이너나 상태

```
kube_pod_container_status_waiting_reason{namespace="app", pod="api"}
```

Grafana 패널 예시

Container Restarts (5m)

Waiting Reason

증거 (kubectl)

Events

Logs (--previous)

실정/의존성 오류 등으로 프로세스 시작 실패

확인 포인트

- 환경변수 / Config
- DB/외부 의존성 연결
- 코드 예외 / 포트 충돌

2 Readiness Probe 실패

Ready 복제본 수

```
sum(kube_deployment_status_replicas_ready{namespace="app", deployment="api"})
```

Service Endpoint 수

```
sum(kube_endpoint_address_available{namespace="app", service="api-svc"})
```

Grafana 패널 예시

Ready Replicas

Service Endpoints

증거 (kubectl)

Events

Logs (--previous)

애플리케이션은 동작하지만 준비되지 않아 트래픽 제외

확인 포인트

- /healthz 응답 상태/지연
- 의존 서비스 상태
- 리소스 부족 및 여부

3 CPU Pressure

CPU 사용률 (Pod)

```
sum(rate(container_cpu_usage_seconds_total{namespace="app", pod="api"}[1m])) by (pod)
```

CPU 제한 대비 사용률 (%)

```
100 * sum(rate(container_cpu_usage_seconds_total{namespace="app", pod="api"}[1m])) by (pod) / sum(kube_pod_container_resource_limits{namespace="app", resource="cpu", unit="core"}) by (pod)
```

Grafana 패널 예시

CPU Usage (cores)

CPU Usage vs Limit (%)

증거 (kubectl)

Events

Logs (--previous)

CPU 한계 근접으로 성능 저하, 스로틀링 발생

확인 포인트

- CPU/메모리 한계 설정
- 트래픽 급증 여부
- 쿼리/로직 최적화 필요

2. 상관관계 메트릭스 (Metric → Evidence → Action)

문제 유형

핵심 지표

kubectl 증거

로그/이벤트

개발자에게 전달할 인계 내용

CrashLoopBackOff

restarts_total 증가, waiting_reason

get events, describe pod

logs --previous

시작 실패, 예외 메시지, 환경/설정/의존성 상태 등 포함

Readiness 실패

ready_replicas ↓, endpoints ↓

get endpoints, get events

logs (healthz), 이벤트

/healthz 실패 사유, 의존성 지연, 네트워크/광-한 이슈

CPU Pressure

CPU 사용률 ↑, % of limit ↑

top pod, describe pod

이벤트 (CPU throttling), 성능 로그

리소스 한계, 스로틀링 여부, 지연 구간/제한 강/제어 정보

3. 빠른 진단 절차 (Runbook)

1 증상 확인: Grafana 대시보드에서 이상 패턴 확인

2 범위 확인: Namespace / Deployment / Pod 식별

3 메트릭 확인: PromQL로 핵심 지표 확인

4 증거 수집: events, describe, logs, top

5 원인 추정 및 조치: 설정/코드/리소스/의존성

6 개발자 인계: 재현 방법, 로그, 설정, 타임라인 포함

4. 개발자 인계 템플릿 (Copy & Paste)

요약

- 문제 상황: <CrashLoopBackOff / Readiness / CPU Pressure>
- 발생 시간: <2023-07-09 15:30 ~ 15:40>
- 영향 범위: <api Deployment, app Namespace>

증거

- 핵심 메트릭: <Grafana 링크>
- 이벤트: <kubectl get events --sort-by:.lastTimestamp>
- 로그: <kubectl logs --previous --pod=api> 또는 healthz 로그
- 추가 정보: <한 줄 변수, Config, 의존성 상태>

재현 방법

- <설정/트래픽 조건>
- <영향 받는 서비스 개요>

5. 베스트 프랙티스

- /healthz, /readyz 명확히 구현
- 리소스 Requests/Limits 적절히 설정
- 의존성 타임아웃/재시도/서킷브레이커 적용
- 로그 포맷(JSON) 및 trace_id 포함
- 알림은 메트릭 기반 + Runbook 연결
- 변경 시 배포 전 검증(테스트/부하/보안)

에트릭은 '무언이 일어났는가'를, 로그와 이벤트는 '왜 일어났는가'를 알려줍니다!

도구 연결 흐름

Prometheus

Grafana

kubectl

Developer

GitHub/PR

GitHub Actions

Production

아이콘 설명

에트릭

로그/이벤트

조치/명령

사용자/개발자

수업 목표

- CrashLoop, readiness 실패, CPU 압박을 metric과 연결한다.
- metric만 보고 결론 내리지 않고 logs/events와 함께 확인한다.
- 장애 시나리오를 PromQL과 dashboard evidence로 남긴다.

장애 시나리오 배포

```
kubectl apply -f week4/day3/labs/observability-scenarios/namespace.yaml
kubectl apply -f week4/day3/labs/observability-scenarios/crashloop-demo.yaml
kubectl apply -f week4/day3/labs/observability-scenarios/cpu-pressure-demo.yaml
kubectl apply -f week4/day3/labs/observability-scenarios/readiness-bad-demo.yaml
```

kubectl 확인:

```
kubectl -n week4-observe get pod
```

예상:

file:///C:/Users/Public/Documents/ESTsoft/CreatorTemp/lecture-pdf-gh8cj9pk/index.html

1/6

```
crashloop-demo-xxxxx      0/1    CrashLoopBackOff
cpu-pressure-demo-xxxxx   1/1    Running
readiness-bad-demo-xxxxx  0/1    Running
```

실제 검증에서는 rollout 중 다음처럼 보일 수 있다.

```
cpu-pressure-demo-...      1/1    Running
crashloop-demo-...         0/1    CrashLoopBackOff
readiness-bad-demo-...     1/1    Running
readiness-bad-demo-...     0/1    Running
```

readiness-bad-demo가 두 개 보이는 것은 새 Pod가 Ready가 되지 못해 rollout이 멈췄고, 기존 Ready Pod가 service 안정성을 위해 남아 있기 때문이다.

CrashLoop과 restart metric

```
kube_pod_container_status_restarts_total{namespace="week4-observe"}
```

최근 5분 증가량:

```
increase(kube_pod_container_status_restarts_total{namespace="week4-observe"}[5m])
```

kubectl 원인:

```
kubectl -n week4-observe logs deploy/crashloop-demo --previous
kubectl -n week4-observe describe pod -l app=crashloop-demo
```

출력 예시:

```
intentional restart for observability
```

event 예시:

```
Back-off restarting failed container
```

restart metric은 “재시작이 늘었다”를 보여주고, log/event는 “왜 재시작했는가”를 보여준다.

readiness 실패

readiness 실패는 restart가 늘지 않을 수 있다.

```
kubectl -n week4-observe describe pod -l app=readiness-bad-demo
```

예상 event:

```
Readiness probe failed: HTTP probe failed with statuscode: 404
```

metric으로는 ready replica 또는 condition 계열을 본다.

```
kube_pod_status_ready{namespace="week4-observe", condition="true"}
```

해석:

증상	restart metric	ready metric
CrashLoop	증가	대개 ready 아님
readiness 실패	증가 안 할 수 있음	ready false
정상 rollout 대기	증가 없음	일시적으로 ready 감소

따라서 restart만 보면 readiness 장애를 놓칠 수 있다.

검증된 PromQL 예시:

```
kube_pod_status_ready{namespace="week4-observe", condition="true"}
```

결과 해석:

```
crashloop-demo      0
cpu-pressure-demo   1
readiness old pod    1
readiness new pod    0
```

운영에서는 “Pod가 1개라도 Ready니까 정상”이라고 보지 않는다. rollout status와 ReplicaSet을 함께 봐야 한다.

```
kubectl -n week4-observe rollout status deploy/readiness-bad-demo
kubectl -n week4-observe get rs,pod -l app=readiness-bad-demo
```

CPU 압박

```
sum by (pod) (rate(container_cpu_usage_seconds_total{namespace="week4-observe",
container!="", image!=""}[2m]))
```

limit 대비 사용량은 dashboard에서 함께 본다. CPU limit은 OOM처럼 바로 죽기보다 latency와 throttling으로 드러날 수 있다.

metric과 원인 연결표

metric 변화	원인 후보	함께 볼 것
restart 증가	CrashLoop, OOMKilled	logs previous, describe
ready 감소	readiness 실패, rollout 실패	endpoints, events
CPU 증가	loop, 부하, throttling	app latency, top pod
memory 증가	leak, cache, batch	OOMKilled, working set
target down	scrape 실패	target error, ServiceMonitor

장애를 한 화면에 연결하기

좋은 장애 분석은 세 줄로 연결된다.

1. 증상: /api 503 증가
2. metric: ready replica 감소, ingress 5xx 증가
3. 원인 evidence: readiness probe 404 event

또는:

1. 증상: 응답 지연
2. metric: api Pod CPU rate 증가
3. 원인 evidence: cpu-pressure-demo loop, resource limit 낮음

metric 이름만 나열하면 개발팀이 움직이기 어렵다. 사용자 증상과 Kubernetes evidence를 함께 전달해야 한다.

시나리오별 관찰 순서

CrashLoop:

```
Grafana restart panel
-> PromQL increase(restarts[5m])
-> kubectl get pod
-> logs --previous
-> describe pod events
```

Readiness 실패:

Grafana ready replica 감소

- > kube_pod_status_ready
- > kubectl get endpoints
- > describe pod readiness event
- > Ingress 503 여부 확인

CPU 압박:

Grafana CPU usage

- > PromQL rate(container_cpu_usage_seconds_total)
- > kubectl top pod
- > resource requests/limits
- > latency/readiness 영향 확인

장애 재현 후 정리

실습 장애는 계속 남겨두면 다음 수업에 방해된다.

```
kubectl delete namespace week4-observe
```

하지만 삭제 전에는 반드시 evidence를 남긴다.

evidence	예시
kubectl	get pod, describe pod, logs --previous
PromQL	restart increase, CPU rate
Grafana	Pod dashboard screenshot
판단	metric과 event가 어떻게 연결되는지

오해하기 쉬운 지점

오해	정리
CPU가 높으면 장애다	지속 시간과 사용자 영향 필요
restart가 있으면 무조건 문제다	rollout 중 일시 restart일 수도 있음
readiness 실패는 log에 항상 보인다	event에 먼저 보일 때가 많음
metric이 없으면 정상이다	target discovery 실패일 수 있음

Evidence Note

W4D3S5 failure correlation

- 재현한 장애:
- kubectl 증상:
- PromQL:
- Grafana dashboard:
- logs/events 원인:
- 개발팀에 전달할 문장:
- 정리/cleanup 여부:

한 줄 요약

metric은 장애의 시간과 범위를 보여주고, logs/events는 원인을 좁히는 증거다.